

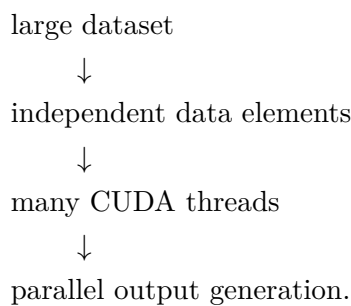
CUDA Chapter 2 Notes: Heterogeneous Data Parallelism

1 Heterogeneous Data-Parallel Computing

Chapter 2 turns the general motivation for GPU computing into a concrete CUDA programming model.

Heterogeneous computing uses different processors for different parts of a program: the CPU acts as the **host**, while the GPU acts as the **device**. **Data parallelism** means applying the same computation to many independent data elements.

The basic mapping is



A simple example is RGB-to-grayscale conversion. For pixel i ,

$$I[i] = (r_i, g_i, b_i),$$

and its grayscale luminance is

$$L_i = 0.21r_i + 0.72g_i + 0.07b_i.$$

Because each output pixel depends only on its corresponding input pixel, different pixels can be processed independently.

Thus, a serial loop

loop iteration i

can often be mapped directly to

CUDA thread i .

1.1 Data Parallelism vs. Task Parallelism

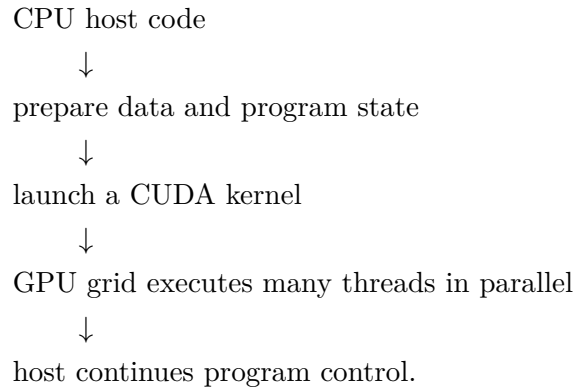
Type	Main idea	Example
Data parallelism	Same operation on many data elements	Process many pixels
Task parallelism	Different independent activities	Overlap transfer and computation

This chapter mainly focuses on **data parallelism**, which provides the large amount of independent work needed for GPU execution.

2 CUDA C Program Structure

A CUDA source file may contain both CPU and GPU code. The host controls program execution and launches kernels; each kernel launch creates a grid of GPU threads.

The basic execution model is



A kernel launch uses CUDA's execution-configuration syntax:

```
kernelName<<<numBlocks, threadsPerBlock>>>(arguments);
```

For a one-dimensional launch,

$$N_{\text{threads}} = N_{\text{blocks}} \times N_{\text{threads/block}}.$$

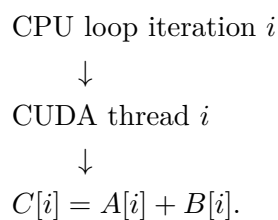
The key point is that CPU execution remains primarily control-oriented, while the kernel represents a parallel phase executed by many GPU threads.

3 Vector Addition: From Loop Iterations to Threads

Vector addition provides the minimal CUDA example:

$$C[i] = A[i] + B[i], \quad 0 \leq i < n.$$

A CPU implementation would normally iterate over i . CUDA replaces these independent loop iterations with independent threads:



The kernel therefore only needs to determine which element belongs to the current thread:

```
__global__
void vecAddKernel(const float* A,
                  const float* B,
                  float* C,
                  int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < n) {
        C[i] = A[i] + B[i];
    }
}
```

```
}
}
```

This kernel contains the two essential ideas of Chapter 2:

- compute a unique global data index for each thread;
- protect memory accesses with a boundary check.

4 Grid, Blocks, Threads, and Thread Indexing

A kernel launch creates a **grid**. The grid contains multiple **thread blocks**, and each block contains multiple **threads**:

```
kernel launch → grid → thread blocks → threads
```

For a one-dimensional grid, CUDA provides three important built-in variables:

Variable	Meaning
<code>threadIdx.x</code>	Thread index inside the current block
<code>blockIdx.x</code>	Block index inside the grid
<code>blockDim.x</code>	Number of threads in each block

The global thread index is

```
i = blockIdx.x × blockDim.x + threadIdx.x
```

For example, if each block contains 256 threads,

Block 0 → $i = 0, \dots, 255$,

Block 1 → $i = 256, \dots, 511$,

Block 2 → $i = 512, \dots, 767$.

Thus, the block index identifies a group of threads, while the thread index identifies one thread inside that group.

Thread blocks within a grid are independent and may execute in any order. This independence allows the same kernel to scale across GPUs with different amounts of hardware.

5 CUDA Function Qualifiers

CUDA extends C/C++ function declarations with qualifiers that specify where a function is called and where it executes.

Qualifier	Callable from	Executes on	Role
<code>__host__</code>	Host	Host	CPU function
<code>__global__</code>	Host	Device	Kernel launching a grid
<code>__device__</code>	Device	Device	GPU helper function

A function without a CUDA qualifier is a host function by default. The most important qualifier in this chapter is `__global__`, because it marks a kernel launched from CPU code and executed by a grid of GPU threads.

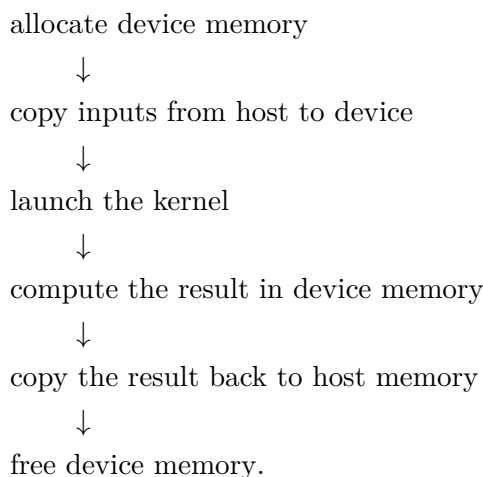
6 Device Global Memory and Data Transfer

Host and device memory are treated as separate memory spaces in the basic CUDA programming model.

A useful naming convention is

$$\begin{aligned} A_h, B_h, C_h &\rightarrow \text{host memory,} \\ A_d, B_d, C_d &\rightarrow \text{device global memory.} \end{aligned}$$

The complete workflow is



The three basic CUDA Runtime API functions are:

Function	Purpose
<code>cudaMalloc</code>	Allocate device global memory
<code>cudaMemcpy</code>	Copy data between host and device
<code>cudaFree</code>	Release device memory

The core syntax is:

```
cudaMalloc((void**)&A_d, size);

cudaMemcpy(A_d, A_h, size,
           cudaMemcpyHostToDevice);

cudaMemcpy(C_h, C_d, size,
           cudaMemcpyDeviceToHost);

cudaFree(A_d);
```

A device pointer such as `A_d` is a variable visible in host code, but the address stored in it refers to GPU memory. Ordinary host code should therefore not dereference it directly.

For n single-precision values,

$$\text{size} = n \times \text{sizeof}(\text{float}).$$

The first argument of `cudaMalloc` is the address of the device-pointer variable because the runtime must write the newly allocated GPU address into that variable.

CUDA API functions return error information. Production code should check these return values, although detailed error-handling code is omitted here to keep the basic workflow clear.

7 Kernel Launch Configuration

For a vector of length n , the host must launch enough CUDA threads so that every valid element can be assigned to one thread.

A standard one-dimensional configuration is

```
int blockSize = 256;
int numBlocks = (n + blockSize - 1) / blockSize;

vecAddKernel<<<numBlocks, blockSize>>>(
    A_d, B_d, C_d, n);
```

Here,

- `blockSize` is the number of threads in each block;
- `numBlocks` is the number of blocks in the grid;
- the launch syntax `<<<numBlocks, blockSize>>>` tells CUDA how many blocks to create and how many threads each block contains.

Therefore, the total number of launched threads is

$$N_{\text{launched}} = N_{\text{blocks}} \times N_{\text{threads/block}}.$$

The grid must contain at least n threads:

$$N_{\text{launched}} \geq n.$$

If each block contains $N_{\text{threads/block}}$ threads, the required number of blocks is

$$N_{\text{blocks}} = \left\lceil \frac{n}{N_{\text{threads/block}}} \right\rceil.$$

Because ordinary C/C++ integer division truncates the fractional part, CUDA programs often implement this ceiling division as

$$\text{numBlocks} = \frac{n + \text{blockSize} - 1}{\text{blockSize}}$$

which appears in code as

```
int numBlocks =
    (n + blockSize - 1) / blockSize;
```

For example, let

$$n = 1000, \quad \text{blockSize} = 256.$$

Then

$$N_{\text{blocks}} = \left\lceil \frac{1000}{256} \right\rceil = 4,$$

so CUDA launches

$$4 \times 256 = 1024$$

threads.

The first 1000 threads correspond to valid vector elements, while the remaining 24 threads have indices outside the data range. This is why the kernel contains a boundary check:

```

if (i < n) {
    C[i] = A[i] + B[i];
}

```

Without this condition, the extra threads could access memory beyond the allocated arrays. The complete indexing logic is therefore

$$i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$$

followed by

$$i < n.$$

The launch configuration and the boundary check work together:

launch enough threads $\rightarrow$ compute a unique global index $\rightarrow$ discard out-of-range threads.

A block size such as 256 is a common practical starting point because it provides many threads per block while remaining within CUDA's block-size limits. The best block size is not universal, however; later chapters relate it to warp organization, resource usage, occupancy, and memory behavior.

8 Minimal Host–Device Vector Addition Workflow

The essential host-side code can be reduced to:

```

float *A_d, *B_d, *C_d;
size_t size = n * sizeof(float);

cudaMalloc((void**)&A_d, size);
cudaMalloc((void**)&B_d, size);
cudaMalloc((void**)&C_d, size);

cudaMemcpy(A_d, A_h, size, cudaMemcpyHostToDevice);
cudaMemcpy(B_d, B_h, size, cudaMemcpyHostToDevice);

int blockSize = 256;
int numBlocks = (n + blockSize - 1) / blockSize;
vecAddKernel<<<numBlocks, blockSize>>>(A_d, B_d, C_d, n);

cudaMemcpy(C_h, C_d, size, cudaMemcpyDeviceToHost);

cudaFree(A_d);
cudaFree(B_d);
cudaFree(C_d);

```

This code is worth remembering as the basic CUDA workflow:

```

cudaMalloc
  ↓
cudaMemcpyHostToDevice
  ↓
kernel launch
  ↓
cudaMemcpyDeviceToHost
  ↓
cudaFree.

```

Vector addition itself performs very little computation per element, so host–device transfer overhead can be significant. In larger CUDA applications, data is often kept on the GPU across multiple kernel launches instead of being copied back after every operation.

9 CUDA Compilation

A CUDA source file usually uses the extension `.cu`. It may contain both ordinary C/C++ host code and CUDA-specific device code.

Ordinary C/C++ compilers do not directly understand all CUDA language extensions, such as

- function qualifiers such as `__global__` and `__device__`;
- built-in thread variables such as `threadIdx` and `blockIdx`;
- CUDA Runtime API calls;
- kernel-launch syntax such as `<<<...>>>`.

CUDA programs are therefore compiled through `nvcc`.

9.1 `nvcc`: NVIDIA CUDA Compiler Driver

`nvcc` stands for the **NVIDIA CUDA Compiler Driver**. It is the compiler driver used to build CUDA programs.

The important point is that `nvcc` does not treat the entire `.cu` file as one ordinary CPU program. Instead, it coordinates the compilation of the two parts of a heterogeneous CUDA program:

CUDA source → host code + device code

The host portion is passed to a normal host C/C++ compiler, while the device portion is processed by NVIDIA’s CUDA compilation toolchain.

For example, the host compiler may be

- Microsoft `cl.exe` on Windows;
- `g++` on many Linux systems.

9.2 PTX

PTX stands for **Parallel Thread Execution**. It is an NVIDIA-defined intermediate instruction representation for GPU programs.

PTX is neither the original CUDA C/C++ source code nor the final machine code executed directly by the GPU. Instead, it is an intermediate representation between high-level CUDA code and hardware-specific GPU instructions.

Conceptually,

CUDA device code $\rightarrow$ PTX $\rightarrow$ GPU machine code

This intermediate form allows the CUDA toolchain and driver to translate device code for a particular GPU architecture.

9.3 JIT Compilation

JIT means **Just-In-Time compilation**. In the simplified compilation path used in this chapter, PTX is converted by the NVIDIA driver into GPU-executable machine code for the GPU that is actually present in the system.

Thus, the main compilation flow is

CUDA C/C++ source (.cu) $\rightarrow$ `nvcc`

$$\rightarrow \begin{cases} \text{host code} \rightarrow \text{host C/C++ compiler} \rightarrow \text{CPU code,} \\ \text{device code} \rightarrow \text{PTX} \rightarrow \text{driver JIT compiler} \rightarrow \text{GPU machine code.} \end{cases}$$

The key idea is that one CUDA source program ultimately contains code for two different processors:

CPU executes host code GPU executes device code

This is another manifestation of heterogeneous computing: CUDA programming is not simply “writing code for a GPU,” but coordinating CPU execution with GPU execution in one application.

9.4 Typical Compilation Command

A minimal CUDA program can be compiled with

`nvcc vec_add.cu -o vec_add`

where

- `vec_add.cu` is the CUDA source file;
- `-o vec_add` specifies the output executable name;
- `nvcc` coordinates the host and device compilation stages.

For Chapter 2, the essential model to remember is

`.cu` $\rightarrow$ `nvcc` $\rightarrow$ host compilation + device compilation

together with the device-side path

CUDA device code $\rightarrow$ PTX $\rightarrow$ GPU-executable code.